GUÍA DE APRENDIZAJE N.º 10. SUBALGORITMOS EN PYTHON: FUNCIONES, PROCEDIMIENTOS Y MODULARIZACIÓN

- Asignatura: Diseño Funcional
- Unidad: 2 – Estructuras de Datos
- Tema: 5. Subalgoritmos
- Fecha: 06 de noviembre de 2026

1. CONCEPTOS CLAVE

1.1. ¿Qué es un subalgoritmo?

Un subalgoritmo es un conjunto de instrucciones que realiza una tarea específica dentro de un programa. Permite dividir un programa extenso en partes más pequeñas, organizadas y reutilizables. En Python, los subalgoritmos se implementan principalmente mediante funciones.

Ventajas:

- Evitan repetir instrucciones.
- Facilitan la organización del código.
- Permiten reutilizar operaciones.
- Simplifican las pruebas y la corrección de errores.
- Favorecen el mantenimiento de los aplicativos.

1.2. Funciones en Python

Una función es un bloque de instrucciones que puede recibir datos, procesarlos y devolver un resultado. Se define mediante la palabra reservada def.

```
def calcular_total(precio, cantidad):
    total = precio * cantidad
    return total

resultado = calcular_total(50000, 3)
```

```
print("Total:", resultado)
```

Elementos principales:

- def: permite definir la función.
- calcular_total: nombre de la función.
- precio, cantidad: parámetros.
- return: devuelve el resultado.
- calcular_total(50000, 3): llamada a la función con argumentos.

1.3. Procedimientos

En programación, un procedimiento realiza una tarea sin necesidad de devolver un resultado. En Python se representa mediante una función que no utiliza return para devolver un valor.

```
def mostrar_producto(nombre, precio):
    print("Producto:", nombre)
    print("Precio:", precio)

mostrar_producto("Teclado", 85000)
```

La función ejecuta las instrucciones, pero no devuelve un resultado para ser utilizado posteriormente.

1.4. Funciones lambda

Una función lambda permite definir operaciones sencillas mediante una expresión breve.

```
calcular_descuento = lambda precio: precio * 0.10

print(calcular_descuento(100000))
```

Resultado:

10000.0

Las funciones lambda son útiles cuando necesitamos operaciones pequeñas, como establecer criterios de ordenamiento.

1.5. Paso de argumentos: valor y referencia

En Python, los argumentos se pasan mediante referencias a objetos. El comportamiento depende de si el objeto es mutable o inmutable.

Ejemplo con un número:

```
def modificar_numero(numero):
    numero = 100

valor = 50

modificar_numero(valor)

print(valor)
```

Resultado:

50

La variable original conserva su valor porque la asignación dentro de la función no modifica el objeto original.

Ejemplo con una lista:

```
def agregar_producto(productos):
    productos.append("Mouse")

inventario = ["Teclado", "Monitor"]

agregar_producto(inventario)

print(inventario)
```

Resultado:

['Teclado', 'Monitor', 'Mouse']

En este caso, la lista original se modifica porque es un objeto mutable.

1.6. Variables locales y globales

Una variable local se define dentro de una función y se utiliza en su ámbito.

```
def calcular_iva(precio):
    iva = precio * 0.19
    return iva

print(calcular_iva(100000))
```

La variable iva es local.

Una variable global se define fuera de las funciones.

```
nombre_empresa = "Mi Empresa"

def mostrar_empresa():
    print(nombre_empresa)

mostrar_empresa()
```

Aunque Python permite trabajar con variables globales, se recomienda utilizar parámetros y valores de retorno para organizar mejor el código.

2. ACTIVIDAD PRÁCTICA

Desarrolla y ejecuta el siguiente programa en Python.

El aplicativo permite registrar productos, consultar el inventario, buscar registros y calcular el valor total mediante funciones y procedimientos.

```
# SISTEMA DE INVENTARIO CON FUNCIONES

productos = []


# REGISTRAR PRODUCTO
```

```
def registrar_producto():
    print("\nREGISTRAR PRODUCTO")

    codigo = input("Código: ").strip()
    nombre = input("Nombre: ").strip()

    for producto in productos:
        if producto["codigo"] == codigo:
            print("El código ya existe.")
            return

    try:
        precio = float(input("Precio: "))
        cantidad = int(input("Cantidad: "))

        if not codigo or not nombre:
            print("Código y nombre son requeridos.")
            return

        if precio < 0 or cantidad < 0:
            print("Los valores no pueden ser negativos.")
            return

        producto = {
            "codigo": codigo,
            "nombre": nombre,
            "precio": precio,
            "cantidad": cantidad
        }
```

```
        productos.append(producto)

        print("Producto registrado correctamente.")

    except ValueError:
        print("Ingrese valores numéricos válidos.")


# CALCULAR VALOR DE UN PRODUCTO

def calcular_subtotal(precio, cantidad):
    return precio * cantidad


# MOSTRAR PRODUCTOS

def mostrar_productos():
    print("\nLISTADO DE PRODUCTOS")

    if not productos:
        print("No existen productos registrados.")
        return

    for producto in productos:
        subtotal = calcular_subtotal(
            producto["precio"],
            producto["cantidad"]
```

```
        print("--------------------")
        print("Código:", producto["codigo"])
        print("Nombre:", producto["nombre"])
        print("Precio:", producto["precio"])
        print("Cantidad:", producto["cantidad"])
        print("Subtotal:", subtotal)


# BUSCAR PRODUCTO

def buscar_producto():
    codigo = input("\nCódigo a buscar: ").strip()

    for producto in productos:
        if producto["codigo"] == codigo:
            print("Producto encontrado:")
            print(producto)
            return

    print("Producto no encontrado.")


# CALCULAR TOTAL DEL INVENTARIO

def calcular_total_inventario():
    total = 0

    for producto in productos:
        total += calcular_subtotal(
```

```
            producto["precio"],
            producto["cantidad"]
        )

    return total


# ORDENAR PRODUCTOS

def ordenar_productos():
    ordenados = sorted(
        productos,
        key=lambda producto: producto["precio"]
    )

    print("\nPRODUCTOS ORDENADOS POR PRECIO")

    for producto in ordenados:
        print(producto["nombre"], "-", producto["precio"])


# MENÚ PRINCIPAL

def mostrar_menu():
    print("\n===== SISTEMA DE INVENTARIO =====")
    print("1. Registrar producto")
    print("2. Mostrar productos")
    print("3. Buscar producto")
    print("4. Calcular total del inventario")
```

```
    print("5. Ordenar productos por precio")
    print("0. Salir")


def iniciar_programa():
    while True:
        mostrar_menu()

        opcion = input("Seleccione una opción: ")

        if opcion == "1":
            registrar_producto()

        elif opcion == "2":
            mostrar_productos()

        elif opcion == "3":
            buscar_producto()

        elif opcion == "4":
            total = calcular_total_inventario()
            print("Valor total:", total)

        elif opcion == "5":
            ordenar_productos()

        elif opcion == "0":
            print("Programa finalizado.")
```

```
            break

        else:
            print("Opción no válida.")


# EJECUTAR PROGRAMA

if __name__ == "__main__":
    iniciar_programa()
```

3. APLICACIÓN AL PROYECTO

Después de ejecutar y comprender el código anterior, continúa con el proyecto de software que vienes desarrollando.

ACTIVIDAD 1. IDENTIFICACIÓN DE SUBALGORITMOS

Revisa el código de tu aplicativo e identifica las operaciones que pueden organizarse mediante funciones, como registrar, consultar, buscar, actualizar o procesar información.

ACTIVIDAD 2. IMPLEMENTACIÓN DE FUNCIONES

Implementa al menos tres funciones que reciban parámetros y devuelvan resultados mediante return, de acuerdo con las necesidades del proyecto.

ACTIVIDAD 3. IMPLEMENTACIÓN DE PROCEDIMIENTOS

Organiza las operaciones de interacción con el usuario mediante funciones que ejecuten tareas sin necesidad de devolver resultados.

ACTIVIDAD 4. APLICACIÓN DE FUNCIONES LAMBDA

Utiliza al menos una función lambda para realizar una operación sencilla, como ordenar registros mediante un atributo o efectuar un cálculo.

ACTIVIDAD 5. VARIABLES Y ARGUMENTOS

Identifica las variables locales y globales utilizadas en tu proyecto. Revisa cómo se comportan los argumentos al trabajar con valores numéricos, cadenas, listas y diccionarios.

ACTIVIDAD 6. MODULARIZACIÓN DEL APLICATIVO

Reorganiza el código existente para evitar instrucciones repetidas, utilizando funciones con responsabilidades específicas. Conserva las funcionalidades desarrolladas anteriormente.

ACTIVIDAD 7. PRUEBAS DE FUNCIONAMIENTO

Ejecuta el aplicativo y comprueba que las funciones implementadas reciben correctamente los argumentos, procesan los datos y producen los resultados esperados.